Name: PRANJAL SWARUP BHATNAGAR Reg. No.: RA2512703010007

Course: 21CSC202J - Operating Systems

Unit – 2 : Session – 17 : SLO – 2 | Dining Philosophers in C — Missing Code

Answers

TODO 1: sem_wait(&forks[id]); (left fork)

TODO 2: sem_wait(&forks[(id + 1) % N]); (right fork)

TODO 3: sem_post(&forks[id]);

TODO 4: sem_post(&forks[(id + 1) % N]);

Completed program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5 // Number of philosophers

sem_t forks[N];

void* philosopher(void* num) {
    int id = *((int*)num);
    printf("Philosopher %d is thinking\n", id);
    sleep(1);
    // Pick up forks
    sem_wait(&forks[id]);                       /* TODO 1 */
    sem_wait(&forks[(id + 1) % N]);             /* TODO 2 */
    printf("Philosopher %d is eating\n", id);
    sleep(2);
    // Put down forks
    sem_post(&forks[id]);                       /* TODO 3 */
    sem_post(&forks[(id + 1) % N]);             /* TODO 4 */
    printf("Philosopher %d finished eating and is thinking again\n", id);
    return NULL;
}

int main() {
    pthread_t threads[N];
    int ids[N];
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1); // Initialize each fork
        ids[i] = i;
    }
    for (int i = 0; i < N; i++)
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    for (int i = 0; i < N; i++)
        pthread_join(threads[i], NULL);
    for (int i = 0; i < N; i++)
        sem_destroy(&forks[i]);
    return 0;
}
```

Deadlock note

1. If all five philosophers pick up their left fork at the same moment, each waits forever
2. for the right fork (circular wait -> deadlock).

3. Fix (asymmetric ordering): the last philosopher picks up the right fork first, e.g.
4. if (id == N - 1) { sem_wait(&forks[(id + 1) % N]); sem_wait(&forks[id]); }
5. else { sem_wait(&forks[id]); sem_wait(&forks[(id + 1) % N]); }
6. Alternative: a counting semaphore 'room' initialised to N - 1 so at most 4 philosophers
7. compete for forks at once.